

Critical Analysis: Tree of Problems: Improving Structured Problem Solving with Compositionality

Kai-Yu Lu

1 Methodological Strengths

1.1 Clear problem framing around compositionality

The study targets structured problems where a complex instance can be decomposed into smaller, analogous subproblems and then recomposed into a final answer. This framing isolates a class of tasks termed *canonical tasks*, contrasted with *sequential tasks* where later steps depend on earlier outputs. The method explicitly operationalizes this framing through a tree parameterization with *breadth* (number of children per internal node) and *depth* (granularity of atomic subproblems), denoted as $\text{ToP}(b, d)$. The design cleanly separates (i) a *decomposer* (problem splitting), (ii) a *solver* (leaf solving), and (iii) a *merger* (bottom-up recomposition), enabling targeted analysis of where performance gains and failures arise.

1.2 A simplified alternative to ToT and GoT with controllable compute

The method is positioned as a simplification of Tree-of-Thought (ToT) and differs by enforcing decomposition into *identical subtasks* and restricting merging prompts to depend only on directly connected child nodes (local composition), rather than requiring global search or graph-level reasoning. Compute is tracked at the level of inference calls: the reported ToP inference cost (excluding decomposition) equals the number of nodes in the constructed tree, and nodes at the same level are solved in parallel. For sequential tasks, setting $b = 1$ yields a depth-wise hierarchy where the model is not required to merge multiple children; instead, the next subproblem is automatically reformulated using the previous solution, and the final answer is obtained at the last subproblem (the root node does not influence inference cost). This explicit protocol strengthens interpretability of compute-performance trade-offs and clarifies the role of depth in sequential reasoning.

1.3 Diverse comparisons and cost-aware fairness checks

The empirical design includes comparisons against Input-Output (direct prompting), Chain-of-Thought (CoT), ToT (best setting), Graph-of-Thoughts (GoT, best setting), Least-to-Most (L2M), and CoT with Self-Consistency. A notably strong element is the fairness analysis for L2M on Last Letter Concatenation: ToP is adapted to match L2M’s number of inference calls, enabling a cost-matched comparison that isolates structural benefits rather than compute inflation.

1.4 Concrete empirical wins on canonical tasks

Across several canonical tasks, ToP provides large absolute improvements over prior structured prompting baselines.

Table 1: Core results reported in the study (Exact String Match accuracy). “N/A” indicates the baseline is not reported for that task setting.

Benchmark Group	Task (Setting)	IO	CoT	ToT/GoT	ToP
GoT tasks (gpt-3.5-turbo)	Sorting	N/A	0.02	0.17 / 0.28	0.68
	Set Intersection	N/A	0.07	0.25 / 0.46	0.65
	Keyword Counting	N/A	0.00	0.00 / 0.26	0.31
Symbolic reasoning (gpt-3.5-turbo-instruct)	Last Letter (L=4)	0.032	0.900	N/A	0.990
	Last Letter (L=8)	0.000	0.662	N/A	0.854
	Last Letter (L=16)	0.000	0.252	N/A	0.444
BBH canonical tasks (gpt-3.5-turbo-instruct)	Hyperbaton	0.528	0.804	N/A	0.902
	Word Sorting	0.837	0.619	N/A	0.831
	Boolean Expressions	0.908	0.924	N/A	0.896
	Multistep Arithmetic Two	0.032	0.780	N/A	0.736
	Object Counting	0.412	0.928	N/A	0.892
BBH sequential tasks (gpt-3.5-turbo-instruct)	Coin Flip (L=4)	0.512	0.998	N/A	0.998
	Coin Flip (L=8)	0.502	0.840	N/A	0.998
	Coin Flip (L=16)	0.476	0.718	N/A	0.756
	Navigate	0.204	0.864	N/A	0.884
	Tracking Shuffled Objects (3)	0.004	0.536	N/A	0.524
	Tracking Shuffled Objects (5)	0.004	0.324	N/A	0.440
	Tracking Shuffled Objects (7)	0.000	0.044	N/A	0.118
	Web of Lies	0.528	0.920	N/A	0.972

The improvements on GoT tasks are particularly pronounced, with Sorting improving from GoT’s 0.28 to ToP’s 0.68, and Set Intersection improving from 0.46 to 0.65. On sequential tasks, Web of Lies improves from CoT 0.920 to ToP 0.972, and Tracking Shuffled Objects improves consistently as the number of objects increases, suggesting that depth-wise decomposition helps maintain correctness under longer interaction chains.

1.5 Helpful diagnostic appendices

The appendices provide actionable diagnostics: (i) sampling and decoding policies (nucleus sampling with $T = 0.7$, top- $p = 0.95$ when sampling is required; greedy decoding with $T = 0.0$ otherwise; up to 2000 new tokens; Exact String Match as the metric), (ii) explicit decomposition implementations (algorithmic split for sorting and set intersection, sentence-based splitting for keyword sorting via a sentence splitter), and (iii) depth and breadth analyses that reveal nontrivial performance cliffs and trade-offs.

2 Key Limitations

2.1 Restricted applicability to structured, decomposable problems

The study explicitly limits applicability to tasks with a structure that can be decomposed into analogous subproblems. This restriction excludes many realistic settings where subproblems are heterogeneous, where decomposition is not obvious, or where intermediate results are not readily mergeable.

2.2 Task-specific decomposition is largely hand-engineered

Although the framework allows algorithmic or prompt-based decomposition, implementation details indicate that all decomposers used are algorithmic. This improves control but reduces generality: performance relies on correct, task-specific decomposition logic (for example, merge-sort-like splits for sorting). As a re-

sult, part of the reported gains can be attributed to injecting algorithmic structure rather than purely eliciting latent reasoning capabilities from a fixed model.

2.3 Performance regressions on several canonical tasks

ToP does not dominate CoT across all BBH canonical tasks. For Boolean Expressions, ToP (0.896) underperforms CoT (0.924). For Multistep Arithmetic Two, ToP (0.736) underperforms CoT (0.780). For Object Counting, ToP (0.892) underperforms CoT (0.928). These regressions indicate that decomposition and merging can introduce additional failure points that offset the benefits of simplifying subproblems.

2.4 Incomplete accounting of end-to-end cost and latency

The reported inference cost explicitly omits decomposition cost. For tasks where decomposition is nontrivial (for example, sentence splitting or more complex structural parsing), ignoring decomposition can materially misstate practical efficiency. In addition, token-level costs are not reported, and therefore the method’s true computational overhead relative to baselines remains partially unspecified.

2.5 Limited reporting of statistical stability

Exact String Match is reported as a point estimate on finite test sets (for example, 100 examples per GoT task and 500 examples per symbolic task). Confidence intervals, variance across random seeds, and sensitivity to prompt phrasing are not systematically reported. Given the study’s own observation that large language models can fail under minor formulation changes, the lack of statistical stability reporting is a methodological weakness.

3 Technical Bottlenecks

3.1 Error propagation through merging and depth

ToP introduces a multi-stage pipeline: errors at leaves or at intermediate merges can cascade to the root. The appendix analysis formalizes an upper bound for how many main problems can be incorrect given leaf error counts:

$$\text{\#incorrect main problems} \leq \left\lceil \frac{m}{k} \right\rceil, \quad (1)$$

where m is the number of incorrect atomic instances (leaves) and k is the number of leaves required to solve a given main instance. This bound highlights a core bottleneck: improving leaf accuracy alone is insufficient if merging is lossy or inconsistent, and conversely, even a perfect merger cannot exceed the constraint imposed by leaf correctness. The study further evaluates an oracle merger and still reports failure cases, reinforcing that leaf-level robustness and merge-level robustness are both necessary.

3.2 Reasoning inconsistency under minor prompt or context changes

The limitations section states that large language models can fail to be robust to minor changes in problem formulation and may fail on problems closely related to ones they can solve. This inconsistency is identified as a typical failure case in Object Counting and Multistep Arithmetic Two. Such brittleness constrains ToP because decomposition changes surface forms and context structure at each level, increasing exposure to prompt sensitivity.

3.3 Breadth-depth trade-offs are not monotonic

Depth and breadth decisions can cause non-monotonic performance changes. The study reports that deeper trees can degrade performance for some canonical tasks (for example, set intersection), while deeper decomposition tends to improve performance for sequential tasks. This creates a tuning bottleneck: selecting (b, d) requires task insight and can significantly change outcomes, limiting deployment without automated selection.

3.4 Mismatch between training-time inductive bias and imposed structure

ToP imposes explicit compositional structure (recursive splitting and recombination). Current general-purpose models are not trained with guarantees of compositional generalization under such explicit re-composition protocols. The observed regressions on arithmetic and counting tasks suggest a mismatch: decomposition can disrupt implicit heuristics that CoT uses successfully, while the merger does not consistently reconstruct global constraints.

4 Research Implications

4.1 Prompted compositionality can outperform search-based reasoning on the right task class

The strong improvements over ToT and GoT on sorting and set intersection suggest that, for canonical tasks, enforcing identical-subtask decomposition and local merging can be more effective than global search or graph-based expansion. This indicates that search complexity is not the only bottleneck; aligning the reasoning protocol with the task’s intrinsic compositional structure can be decisive.

4.2 Benchmarks reveal a gap between structured decomposition and robust reasoning

The mixed performance on BBH canonical tasks implies that success requires more than breaking a problem into smaller pieces. In particular, tasks that appear decomposable at a surface level (arithmetic, counting) still require consistent constraint propagation. This points to a broader field-level conclusion: current models do not reliably preserve invariants across multi-step recomposition, even when subproblems are individually simpler.

4.3 Evaluation protocols should separate decomposition, solving, and merging failures

ToP’s modular pipeline implies that evaluation should attribute errors to (i) decomposition correctness, (ii) leaf solving, and (iii) merge consistency. The study takes initial steps via oracle-merger and depth analyses, but the results suggest that future benchmarks should explicitly measure these components to avoid conflating them into a single accuracy number.

5 Potential Research Directions

5.1 Automated and learned decomposition

A concrete direction is to replace hand-engineered decomposers with learned or model-assisted decomposition policies that optimize downstream accuracy under a fixed inference budget. This includes:

- Learning task-conditioned split rules that minimize merge ambiguity.

- Training decomposition selectors to choose (b, d) adaptively per instance, informed by uncertainty or estimated subproblem difficulty.

5.2 Merger verification and constraint-aware recomposition

Given merge-stage brittleness and non-monotonic depth effects, merger design can be strengthened by adding explicit verification:

- Consistency checks that validate merged outputs against child outputs and task constraints.
- Dual-pass merging (compose then verify) where a verifier flags contradictions and triggers localized re-solving.

5.3 Budgeted inference with token-level accounting

Future work should incorporate full cost accounting, including decomposition, token usage, and latency, to assess practical efficiency. Budgeted variants can allocate compute unevenly across levels, prioritizing uncertain subtrees while keeping total cost fixed.

5.4 Robustness protocols for prompt sensitivity

Since minor formulation changes can break performance, robustness-oriented evaluation and prompting protocols are needed:

- Prompt perturbation tests across decomposition levels to quantify stability.
- Canonicalization of intermediate representations to reduce surface-form variance.

6 Conclusion

The Tree of Problems framework provides a clean and modular prompting strategy for structured problem solving by enforcing compositional decomposition into analogous subproblems and bottom-up recomposition. Methodological strengths include controllable breadth-depth structure, parallelizable inference, and strong empirical gains on canonical tasks such as sorting and set intersection, where ToP surpasses ToT and GoT by large margins. Key limitations include restricted task applicability, reliance on hand-engineered decomposition, incomplete end-to-end cost accounting, and measurable regressions on several BBH canonical tasks. The technical bottlenecks center on error propagation through multi-level merging and model inconsistency under minor formulation changes, which together constrain reliability. The most promising research directions are automated decomposition, constraint-aware and verifiable merging, and robustness and cost protocols that explicitly separate and measure decomposition, solving, and recomposition failures.